

ADVENT: LLM-Driven Automatic Predicate Invention for ILP

Tingting Yu

Department of Information Management
National Sun Yat-Sen University
Kaohsiung, Taiwan
tingyui0213@gmail.com

Pei-Cing Huang

Department of Information Management
National Sun Yat-Sen University
Kaohsiung, Taiwan
pcpeicing@gmail.com

Chan Hsu

Department of Information Management
National Sun Yat-Sen University
Kaohsiung, Taiwan
chanshsu@gmail.com

Chan-Tung Ku

Department of Information Management
National Sun Yat-Sen University
Kaohsiung, Taiwan
kuchantung@gmail.com

Yihuang Kang

Department of Information Management
National Sun Yat-Sen University
Kaohsiung, Taiwan
ykang@mis.nsysu.edu.tw

Abstract—Predicate invention (PI), the creation of new predicates to extend the hypothesis space, remains a critical bottleneck in Inductive Logic Programming (ILP). Existing methods rely on domain expertise and produce semantically opaque predicates, hindering adaptation to unfamiliar domains and cross-task reuse. We present ADVENT, an LLM-driven PI mechanism for ILP. ADVENT pairs LLM abductive generation with Prolog deductive verification, forming an iterative loop in which concrete execution results guide the LLM to refine candidate predicates. The mechanism leverages Large Language Models to identify implicit patterns in structured relational data and invent auxiliary predicates with meaningful names and definitions. Invented predicates and learned rules accumulate in a knowledge pool for cross-task reuse. Experiments on nine poker-hand concepts across seven LLMs show that LLM-driven PI achieves 58% success rate where ILP alone fails entirely, formal verification raises this to 80%, and the knowledge pool yields gains up to +31 percentage points, while producing human-interpretable rules. These results suggest that ADVENT offers a promising direction for automating predicate invention and enabling cross-task knowledge reuse in ILP.

Index Terms—Relational Rule Learning, Inductive Logic Programming, Predicate Invention, Large Language Model

I. INTRODUCTION

Many real-world concepts are inherently relational: a molecule is mutagenic not because of any single atom, but because of how atoms are bonded together. Inductive Logic Programming (ILP) [1] is one of the few machine learning paradigms capable of learning such relational concepts [2] as human-interpretable logical rules. However, ILP’s performance depends heavily on predefined background knowledge (BK) [1], a set of predicates that encode known facts and relational structures about the domain, analogous to features in traditional ML. Without suitable predicates, the system cannot discover rules that explain the target concept well. Predicate invention (PI) addresses this limitation by introducing auxiliary predicates that extend the hypothesis space beyond the original BK, making it one of the most critical open challenges in ILP [1]. Yet existing PI approaches suffer from two fundamental limitations. First, without human-provided specifications to guide the search, PI devolves into exhaustive syntactic enumeration over an explosive space, making it

difficult to automatically discover helpful patterns [1], [3]. Second, existing PI methods operate through purely syntactic symbol manipulation and lack conceptual understanding of what they invent [4]. Unnamed predicates such as *inv_1* and *inv_2* have no notion of the patterns these symbols encode, making it hard to assess which prior inventions are relevant to a new task. Furthermore, predicates that build upon earlier inventions become increasingly difficult to interpret. Together, these limitations restrict ILP’s ability to learn in new domains and hinder its potential for lifelong learning [5].

LLMs offer a promising path to address these limitations. Trained on large-scale corpora, LLMs encode broad prior knowledge and demonstrate strong capabilities in code generation, abductive reasoning [6], [7], and logic programs synthesis [8], suggesting they could inspect structured data, recognize implicit patterns, and operate directly within ILP’s symbolic formalism, opening new possibilities for automating predicate invention. Moreover, because LLMs generate predicates with natural language understanding, the invented predicates inherently carry meaningful names (e.g., *same_color*) and interpretable definitions, providing a basis for cross-task relevance assessment.

However, relying solely on LLM reasoning introduces reliability concerns [9]. In preliminary experiments, we observe two recurring types of errors: syntactic violations of logic programming conventions, and predicates that capture overly specific patterns that fail to generalize. Recent studies have shown that iteratively refining LLM outputs with formal verification can substantially improve their reasoning quality [10], [11]. In the context of ILP, Prolog [12], the predominant logic programming language used in ILP, offers a deductive engine that serves as a natural verifier: by executing candidate predicates on subsets of examples, it precisely reveals whether an invented predicate generalizes as intended. These observations motivate three research questions:

- RQ1. Can LLMs serve as an effective mechanism for predicate invention in ILP?
- RQ2. Can formal deductive verification effectively guide LLM predicate refinement and improve ILP learning

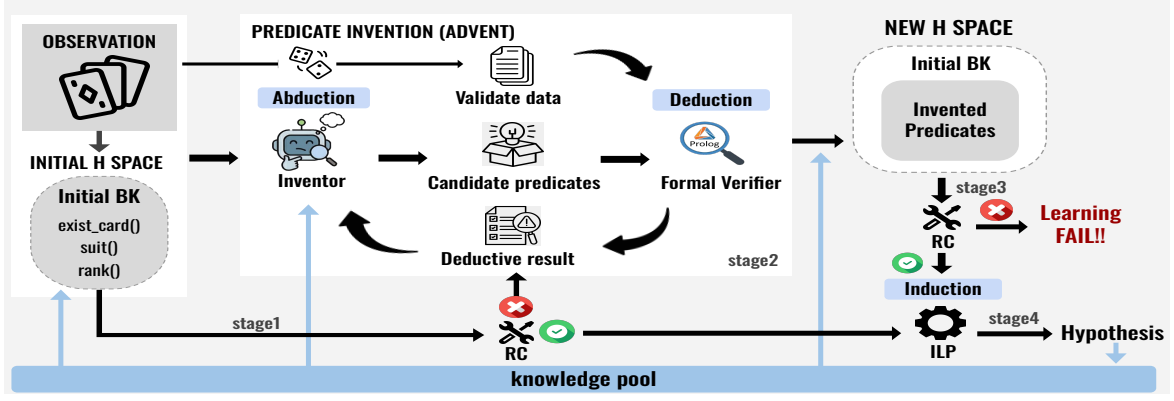


Fig. 1. Overall pipeline.

performance?

- RQ3. Can LLM-driven PI support cross-task predicate reuse and composition while preserving interpretability?

To address these questions, we present ADVENT, an LLM-driven PI mechanism that integrates LLM abductive reasoning with Prolog deductive verifier, and combines them with ILP inductive learning. Invented predicates and learned hypotheses accumulate in a knowledge pool, enabling reuse of prior knowledge across tasks. Our main contributions are as follows:

- We propose ADVENT, an LLM-driven PI mechanism that identifies implicit patterns in structured data and generates corresponding predicates for ILP.
- We integrate a Prolog-based formal verifier that provides concrete deductive results to guide predicate refinement.
- We show that LLMs can interpret invented predicate definitions and assess their relevance to new tasks, enabling selective reuse and composition that supports cumulative learning with interpretable rules.

II. BACKGROUND

We briefly review ILP and relevant prior work to situate ADVENT within the existing literature. An ILP task takes three inputs: background knowledge (BK), which encodes known facts and relational structures about the domain; positive examples (E^+); and negative examples (E^-) of the target concept. The goal is to find a hypothesis (H), a set of first-order logic rules expressed through predicates, variables, and quantifiers, that maximizes coverage of E^+ while minimizing coverage of E^- . This representation is expressive, logically verifiable, and human-interpretable, making ILP well-suited to scientific discovery and safety-critical domains [1]. However, PI within this formalism has been a long-standing challenge [1]. Meta-Interpretive Learning (MIL) systems such as Metagol [13] employ metarules as templates to guide the invention of new predicates, enabling the construction of recursive and compositional hypotheses. However, the expressiveness of Metagol is bounded by the metarules provided, requiring domain expertise to design appropriate templates. Popper [3] takes a different approach by using hypothesis constraints derived from failed hypotheses to prune the search space, supporting PI

through mode declarations that specify the permitted predicate signatures. While more flexible than template-based methods, Popper still relies on manually defined mode declarations to control the search.

These limitations have motivated recent work on integrating LLMs into the ILP pipeline. Yang et al. [14] proposed a multi-agent LLM framework to automatically build BK and language bias from natural-language descriptions of data attributes, substantially reducing manual BK design. However, the generated predicates correspond only to explicit surface-level patterns; implicit relations requiring cross-example reasoning remain outside their scope. Our work targets such implicit patterns, using LLMs not to extract what is described but to invent what is not directly observable. While LLMs can generate logic programs, their outputs in formal domains remain prone to errors [15]. Recent work has shown that iteratively verifying and refining LLM outputs with formal tools can substantially improve correctness [10], [11]. We apply this paradigm using Prolog’s deductive engine: candidate predicates are executed against training examples, and grounded results guide iterative refinement.

Beyond improving invention, a practical PI mechanism must support knowledge reusability across tasks. ILP’s compositional structure naturally supports this: hypotheses learned from earlier tasks can serve as BK for subsequent ones [1]. However, as BK grows, the system faces the relevance problem [1]: without a mechanism to judge which prior knowledge applies to a new task, accumulation leads to catastrophic remembering [5], where irrelevant BK degrades both search efficiency and learning performance. Forgetgol [5] mitigates this through selective forgetting based on usage frequency, but this remains a structural heuristic that does not address the root cause. Our approach addresses the problem at its source: by generating predicates with meaningful names and interpretable definitions, LLM-driven PI produces knowledge that the LLM can later assess for relevance to new tasks.

III. METHODOLOGY

We propose ADVENT, a predicate invention mechanism that pairs LLM-based abductive generation with Prolog-based

deductive verifier. This mechanism is embedded in a neuro-symbolic pipeline that additionally employs the ILP system for rule induction, mirroring Peirce’s triadic reasoning cycle of abduction, deduction, and induction [16]. Fig. 1 illustrates the overall pipeline, which operates in four stages for each learning task.

- *Stage 1: Initial Hypothesis Space Representation Check.* At the onset of learning, the framework first evaluates whether the initial BK already provides sufficient representational power. We introduce a lightweight heuristic, termed the Representation Check (RC), to make this determination (detailed in Section III-A). If RC passes, the framework skips predicate invention and proceeds directly to Stage 4.
- *Stage 2: LLM-based Predicate Invention Loop.* When RC fails, it signals that the available BK lacks the expressiveness to capture the underlying patterns, and the framework enters an iterative invention loop. In this loop, the LLM automatically synthesizes new auxiliary predicates to enrich the hypothesis space, while Prolog provides deductive verification to guide refinement.
- *Stage 3: Post-invention Representation Check.* After PI, the framework re-applies RC over the expanded hypothesis space to validate the effectiveness of the invented predicates. If RC passes, the framework proceeds to rule induction; otherwise, the case is recorded as a learning failure.
- *Stage 4: ILP Rule Induction.* With an adequate hypothesis space confirmed, the ILP learner induces a symbolic hypothesis that generalizes across the provided examples.

In addition to this per-task pipeline, a cross-task mechanism accumulates knowledge across learning episodes. Invented predicates that contribute to successful induction, along with learned rules, accumulate in a knowledge pool that expands the initial hypothesis space of new targets and provides the LLM with a reference for reuse or composition during PI. The following sections detail each component in turn.

A. Representation Check (RC)

To determine the necessity of PI, we introduce RC, a lightweight ILP with restricted variable-level search. Variable-level search replaces specific attribute values with shared variables, forcing the learner to capture relational patterns

across objects. For example, in Fig. 2, the most specific rule for train 2 would be $shape(Car1, S) \wedge shape(Car2, S)$, where the shared variable S captures “same shape” without committing to a specific value. RC operates in three steps: (1) 10% of the positive and negative examples are sampled; (2) 1% of the positive examples from this subset are selected as seeds, each serving as the starting point for an independent rule search; (3) if all trials yield only a ground unit clause that covers solely its seed example with no other positive examples, RC signals that the current BK lacks the predicates needed to express the underlying relational patterns and triggers PI. The 10% example sample and 1% seed selection were chosen as lightweight heuristics to keep RC inexpensive while still checking whether any seed yields a non-trivial variable-level generalization.

B. LLM-PI Loop: Iterative Predicate Invention with Deductive Verification

Once the framework determines via RC that the initial BK is insufficient, this module implements the core predicate invention cycle, in which the LLM abductively proposes candidate predicates and Prolog deductively verifies them to support closed-loop refinement.

1) *LLM Predicate Generation:* At each iteration, the LLM receives a structured prompt consisting of two components: (a) the current task’s BK schema along with a sample of positive and negative examples, and (b) the full inventory of the knowledge pool, listing previously invented predicates with their names and definitions. Given this context, the LLM is prompted to identify implicit patterns across examples and propose one or more candidate predicates, each with a Prolog definition, its arity, and usage specifications such as argument types and input/output modes for integration into the ILP induction system. These predicates may compose directly from the initial BK or, by assessing relevance from their names and definitions, build upon prior inventions to form higher-level abstractions.

2) *Formal Deductive Verification:* After the LLM proposes candidate predicates, Prolog serves as a deductive verifier through two layers of validation. The first layer performs syntactic and structural checks: Prolog attempts to consult the predicate definition, catching runtime errors, unsatisfiable clauses, and undefined predicates. Candidates that fail receive the specific error message for immediate correction. The second layer evaluates predicate behavior on examples. A subset of examples is sampled from both E^+ and E^- , and each candidate is executed via Prolog’s deductive engine. As illustrated in Fig. 2, a unary predicate such as $exist_red_car(Train)$ produces a Boolean result per example, while a higher-arity predicate such as $train_shape(Train, S)$ returns satisfying value tuples. In both cases, results are partitioned by class (E^+ vs. E^-) and presented contrastively, enabling the LLM to assess whether the pattern is discriminative or requires refinement. A maximum iteration limit is enforced to prevent excessive computation.

Target: eastbound train

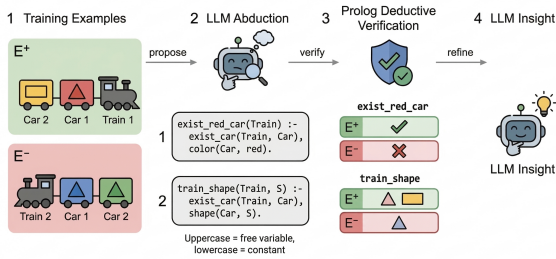


Fig. 2. Illustration of formal deductive verification.

C. ILP Rule Induction

Once RC confirms that generalizable patterns exist, the framework proceeds to rule induction. The auxiliary predicates used to describe these patterns are injected into the original BK to form the refined search space. Since RC has already verified that relational patterns exist at the variable level, the ILP learner then performs a constant-level search, grounding variables to specific values. For example, the same train 2 in Fig. 2 would produce $\text{shape}(\text{Car1}, \text{triangle}) \wedge \text{shape}(\text{Car2}, \text{triangle})$. This allows the learner to capture instance-level distinctions that variable-level rules abstract away, complementing the LLM’s high-level inventions with fine-grained, data-driven rules.

IV. EXPERIMENT

A. Dataset and Task Transfer

We evaluate the proposed framework on a relational concept learning task using the Poker Hand dataset from the UCI Machine Learning Repository [17]. Each instance is a five-card hand with two attributes per card (suit and rank), labeled by a category such as one pair, flush, or straight. To prevent the LLM from exploiting prior knowledge of poker rules, we transfer the task into a classic ILP benchmark, Michalski Train problem [18]: each hand is mapped to a train, each card to a car, suit to shape, rank to load number, and each poker-hand category to a distinct train direction (e.g., one pair as eastbound, two pairs as westbound), preserving the relational structure while making the concepts unfamiliar to the LLM. The initial BK consists of three base predicates: $\text{exist_car}/2$, $\text{shape}/2$, and $\text{load_num}/2$. For each target concept, negative examples are drawn from all lower-ranked hands. To verify this transfer, we prompted the LLM to identify the scenario behind the data; the model consistently interpreted it as a Michalski’s Trains puzzle with no recognition of the poker domain. Conversely, replacing suit and rank predicates with generic variable names such as X_1 and X_2 led the model to immediately recognize the poker scenario, confirming that the transfer effectively abstracts away domain-specific cues.

B. Experimental Setup

We adopt Aleph [19] as the ILP engine, a top-down search system without native PI support. The ILP-only baseline induces over the base BK under two modes: position-dependent, a flattened representation where each card occupies a fixed slot, and position-free, a fully relational representation where cards are accessed through logical relations without fixed positions, better reflecting real poker scenarios. Three additional settings combine LLM-driven PI with Aleph under position-free mode, differing in verification: no verification (single-pass invention), LLM self-critique, and formal verification (ADVENT). Models were accessed through OpenRouter using provider-specific identifiers, e.g., *openai/gpt-5-codex* and *x-ai/grok-4.1-fast*. All three operate under a sequential learning protocol with the knowledge pool, where concepts are learned sequentially from one pair to royal flush in ascending complexity. To isolate the effect of knowledge accumulation, we

TABLE I
LEARNING SUCCESS RATES ACROSS EXPERIMENTAL CONDITIONS.[†]

ILP-only position-dependent		6/9		
ILP-only position-free		0/9		
LLM PI (position-free) + ILP				
Model	No Verif.	LLM Crit.	ADVENT	ADVENT(−KP)
GPT-5-Codex	84% [71–92%]	80% [66–89%]	82% [69–91%]	71% [57–82%]
grok-4.1-fast	64% [50–77%]	78% [64–87%]	78% [64–87%]	73% [59–84%]
gemini-2.5-pro	36% [23–50%]	80% [66–89%]	76% [61–86%]	71% [57–82%]
qwq-32b	62% [48–75%]	49% [35–63%]	67% [52–79%]	53% [39–67%]
GLM-5	78% [64–87%]	73% [59–84%]	80% [66–89%]	80% [66–89%]
MiMo-V2-Omni	31% [20–46%]	80% [66–89%]	93% [82–98%]	56% [41–69%]
deepseek-r1	51% [37–65%]	73% [59–84%]	87% [74–94%]	64% [50–77%]
Avg.	58% [53–63%]	73% [68–78%]	80% [76–84%]	67% [62–72%]

[†]Wilson score intervals are computed over 45 trials (5×9 concepts) per cell; since concepts vary in difficulty, they characterize aggregate performance stability rather than a single underlying rate.

include a formal verification setting without the knowledge pool. Each LLM-based setting is repeated for five runs across nine concepts (45 trials per model). During each PI iteration, the LLM receives five positive and two negative examples per concept as input, with at most three iterations per trial. A trial is counted as successful only when the framework induces the correct ground-truth rule.

V. RESULTS AND DISCUSSION

A. Main Results

As Table I shows, ILP alone learns at most 6 out of 9 concepts under position-dependent binding, and fails entirely without it (0/9). In contrast, even without verification, LLM-driven PI under position-free mode achieves an average success rate of 58% [53–63%], demonstrating that LLMs can identify implicit patterns and invent useful predicates that substantially extend ILP’s representational capacity (RQ1). Adding formal verification (ADVENT) raises the average to 80% [76–84%], whose confidence interval does not overlap with the no-verification condition, confirming that Prolog deductive verification effectively guides predicate refinement (RQ2). By comparison, LLM self-critique achieves 73% [68–78%], suggesting that grounded execution results provide more reliable refinement signals than LLM self-assessment alone. Furthermore, removing the knowledge pool reduces the average from 80% to 67% [62–72%], with drops in six out of seven models and non-overlapping confidence intervals, indicating that the LLM’s ability to reuse previously invented predicates substantially benefits learning (RQ3). The following subsections examine each research question in detail. For readability, all rules and predicates in the following section are presented using the original poker terminology.

B. LLM-Driven Predicate Invention

Table II presents per-concept learning results for ILP alone and ADVENT+ILP, and Table III further illustrates this gap through representative examples. The two ILP-only modes differ in how cards are accessed. Position-dependent mode uses predicate $\text{has_card}(+Hand, -C1, -C2, -C3, -C4, -C5)$,

TABLE II
PER-CONCEPT LEARNING RESULTS.

	1P	2P	3K	ST	FL	FH	4K	SF	RF
ILP-only (pos.)	✓	✓	✓	×	✓	✓	✓	×	×
ILP-only (free.)	×	×	×	×	×	×	×	×	×
ADVENT+ILP (+KP)	63	71	91	69	91	89	91	94	63
ADVENT+ILP (−KP)	57	69	71	60	100	66	83	63	34

pos. = position-dependent, free. = position-free; values in %.
 1P = one pair, 2P = two pairs, 3K = three of a kind;
 ST = straight, FL = flush, FH = full house, 4K = four of a kind;
 SF = straight flush, RF = royal flush.

where $C1$ through $C5$ correspond to fixed card slots. Position-free mode uses $exist_card(+Hand, -C)$, which retrieves an arbitrary card without assuming any ordering. Consider one pair, which requires two cards of the same rank. In position-dependent mode, ILP alone must enumerate all $\binom{5}{2} = 10$ slot combinations, producing a verbose rule set. In position-free mode, ILP alone fails entirely: comparing two cards drawn via $exist_card$ requires an inequality constraint $C1 \neq C2$ to ensure distinct card references, yet ILP alone cannot introduce such constraints autonomously.

The LLM, by contrast, naturally incorporates it when inventing $has_duplicate_rank$, thereby yielding the compact rule: $one_pair(A) :- has_duplicate_rank(A)$. The gap widens with straight, which requires five cards in consecutive ranks. Detecting this pattern demands cross-object arithmetic operations that ILP’s refinement operators cannot construct without pre-defined building blocks. The LLM addresses this by composing three auxiliary predicates min_rank , max_rank , and $has_consecutive_ranks$, while ILP’s subsequent search discovers an additional rule covering the A-10-J-Q-K edge case, illustrating a complementary division of labor. These examples reveal two categories of representational gaps that LLM-driven PI bridges: implicit relational constraints (such as variable inequality) and arithmetic relations (such as consecutive sequences), both fundamentally inaccessible to syntax-driven search.

C. Effectiveness of Formal Deductive Verification

Having characterized the patterns LLMs can identify, we turn to the role of formal verification in refining these inventions. As Table I shows, formal verification improves success rates for six out of seven models. The benefit is most pronounced for models with lower baselines: MiMo rises from 31% to 93% and DeepSeek from 51% to 87%, with non-overlapping confidence intervals in both cases. Models that already generate near-optimal predicates in a single pass (GPT-5-Codex, GLM-5) show marginal gains due to ceiling effects. These results confirm that Prolog deductive verification effectively guides predicate refinement and improves ILP learning performance.

D. Formal Verification vs. LLM Self-Critique

We compare formal verification against LLM self-critique as an alternative refinement signal. While LLM self-critique also

TABLE III
LEARNED RULES FOR ONE PAIR AND STRAIGHT.

ILP-only (pos.)		
1P		one_pair(Hand) :- has_card(Hand,C1,C2,C3,C4,C5), rank(C1,G), rank(C2,G).
		one_pair(Hand) :- has_card(Hand,C1,C2,C3,C4,C5), rank(C1,G), rank(C3, G)..... (C(5,2) = 10 position-specific clauses)
ADVENT+ILP		
1P	PI	has_duplicate_rank(Hand) :- exist_card(Hand, C1), exist_card(Hand, C2), C1 \= C2, rank(C1, R), rank(C2, R).
	ILP	one_pair(Hand) :- has_duplicate_rank(Hand).
ST	PI	min_rank(Hand, Min) :- exist_card(Hand, C1), rank(C1, Min), \+((exist_card(Hand, C2), rank(C2, M), M < Min)).
		max_rank(Hand, Max) :- exist_card(Hand, C1), rank(C1, Max), \+((exist_card(Hand, C2), rank(C2, M), M > Max)).
	ILP	has_consecutive_ranks(Hand) :- min_rank(Hand, Min), max_rank(Hand, Max), Max =:= Min + 4, \+ has_duplicate_rank(Hand).
		straight(Hand) :- has_consecutive_ranks(Hand). straight(Hand) :- exist_card(Hand,C1), rank(C1,10), exist_card(Hand,C2), rank(C2,11), exist_card(Hand,C3), rank(C3,12), exist_card(Hand,C4), rank(C4,13), exist_card(Hand,C5), rank(C5,1).

improves over no verification on average, it is less reliable: QWQ-32B drops from 62% to 49%, suggesting that unreliable self-assessment can mislead refinement. Formal verification outperforms LLM self-critique in five of seven models, with the exception of Gemini-2.5-Pro, where the model’s strong self-assessment partially compensates for its weak baseline. Fig. 3 illustrates this difference on the one pair task. Under formal verification, the LLM initially proposes a predicate checking shared suit, but Prolog’s per-example results immediately flag false positives among negative examples. In the next iteration, a higher-arity predicate retrieves suit-rank pairs for each example, and the contrastive layout makes the duplicate-rank pattern in positive examples directly visible. Under LLM self-critique, by contrast, the model lacks this per-example granularity and progressively overfits to superficial features across iterations, ultimately failing to capture the underlying pattern. This highlights that evaluating predicate behavior on concrete data is inherently a deduction task, and Prolog provides this with formal correctness.

E. Cross-Task Predicate Reuse and Composition

We now examine whether LLM-driven PI enables effective cross-task reuse. Table II breaks down the impact by concept. The largest gains concentrate in compositional concepts (straight flush +31, royal flush +29, full house +23), which leverage knowledge from earlier learning. However, for simpler tasks such as flush (−9), where only a single shared attribute is needed, exposing prior predicates appears to introduce unnecessary complexity. The drop on flush suggests

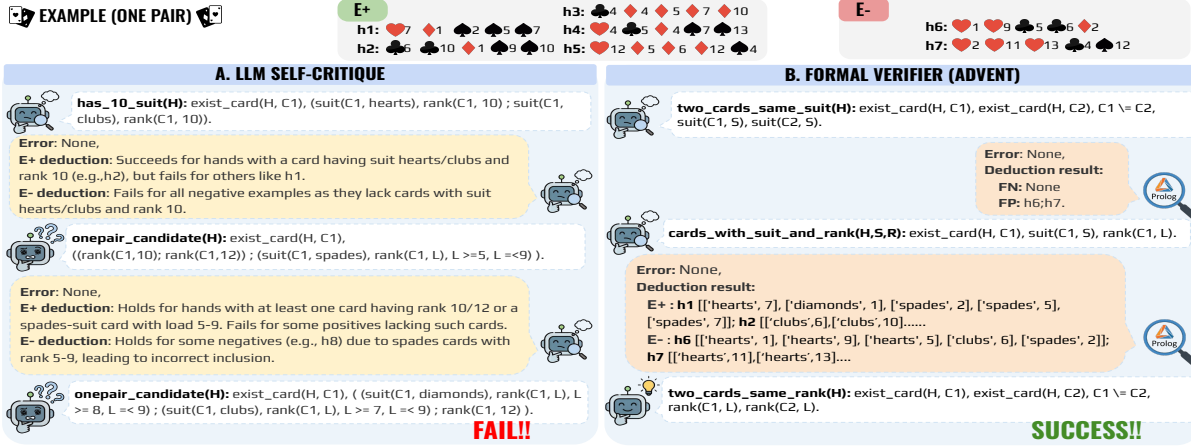


Fig. 3. Case study comparing formal verification and LLM self-critique on the one pair task.

that an unfiltered knowledge pool can introduce distracting predicates for simple concepts, indicating a trade-off between compositional reuse and irrelevant-context interference. As shown in Table III, the predicate *has_consecutive_ranks*, invented during learning straight, reuses the previously invented *has_duplicate_rank* from one pair to exclude hands with repeated ranks. Such cross-task composition arises because the LLM can interpret prior predicate definitions and assess their relevance to the current task. This compositionality extends further: the hypothesis for straight flush composes *has_all_same_suit* (from flush) and *has_consecutive_ranks* (from straight), yet remains immediately comprehensible, maintaining ILP’s core advantage of interpretability.

VI. CONCLUSION

This paper presents ADVENT, an LLM-driven automatic predicate invention mechanism for ILP that integrates LLM abductive reasoning with Prolog deductive verification. Experiments demonstrate that LLMs can identify implicit patterns and invent auxiliary predicates that extend ILP’s representational capacity, achieving a 58% success rate where ILP alone fails entirely. Formal deductive verification raises this to 80%, confirming that grounded execution results effectively guide predicate refinement. The knowledge pool further benefits compositional concepts, with gains up to +31 percentage points, demonstrating that LLM-driven PI supports cross-task reuse and composition while preserving interpretability. However, the current design provides the full pool to the LLM without explicit filtering, relying on the model’s ability to interpret predicate definitions and assess their relevance; as the pool grows, this approach is bounded by the LLM’s context window, and an increasing number of predicates may dilute the model’s attention. In addition, the current evaluation is limited to a transformed poker-hand domain. Future work will develop selective retrieval mechanisms to filter the pool based on relevance and evaluate ADVENT on broader real-world relational datasets.

REFERENCES

- [1] A. Cropper and S. Dumančić, “Inductive logic programming at 30: A new introduction,” *arXiv*, 2022, accessed: Nov. 09, 2024. [Online]. Available: <http://arxiv.org/abs/2008.07912>
- [2] D. M. Gabbay *et al.*, Eds., *Logical and Relational Learning*, ser. Cognitive Technologies. Berlin, Heidelberg: Springer-Verlag Berlin Heidelberg, 2008.
- [3] A. Cropper and R. Morel, “Learning programs by learning from failures,” *Machine Learning*, vol. 110, no. 4, pp. 801–856, Apr. 2021.
- [4] E. Gentili, T. Ribeiro, F. Riguzzi, and K. Inoue, “Predicate renaming via large language models,” *arXiv*, 2025.
- [5] A. Cropper, “Forgetting to learn logic programs,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, Apr. 2020, pp. 3676–3683.
- [6] M. Chen, J. Tworek, H. Jun *et al.*, “Evaluating large language models trained on code,” *arXiv*, 2021.
- [7] Y. Peng, Y. Liu, E. Xia *et al.*, “Abductive logical rule induction by bridging inductive logic programming and multimodal large language models,” *arXiv*, 2025.
- [8] J. ao Pedro Gandarela, D. S. Carvalho, and A. Freitas, “Inductive learning of logical theories with LLMs: An expressivity-graded analysis,” *arXiv*, 2025.
- [9] Z. Ji, N. Lee, R. Frieske *et al.*, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 248:1–248:38, 2023.
- [10] C. Sun, Y. Sheng, O. Padon, and C. Barrett, “Clover: Closed-loop verifiable code generation,” in *AI Verification: First International Symposium, SAIV 2024, Montreal, QC, Canada, July 22–23, 2024, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2024, pp. 134–155.
- [11] V. Singh, D. Cassel, N. Weir, N. Feng, and S. Bayless, “VERGE: Formal refinement and guidance engine for verifiable LLM reasoning,” *arXiv*, 2026.
- [12] P. Körner, M. Leuschel, J. ao Barbosa, V. S. Costa, V. Dahl, M. V. Hermenegildo, J. F. Morales, J. Wielemaker, D. Diaz, S. Abreu, and G. Ciatto, “Fifty years of Prolog and beyond,” *arXiv*, 2022.
- [13] S. H. Muggleton, D. Lin, and A. Tamaddoni-Nezhad, “Meta-interpretive learning of higher-order dyadic datalog: Predicate invention revisited,” *Machine Learning*, vol. 100, no. 1, pp. 49–73, Jul. 2015.
- [14] Y. Yang, J. Wu, and Y. Yue, “Hypothesis generation via LLM-automated language bias for ILP,” *arXiv*, 2026.
- [15] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, “Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning,” *arXiv*, 2023.
- [16] K. He and Z. Chen, “From reasoning to learning: A survey on hypothesis discovery and rule learning with large language models,” *arXiv*, 2025.
- [17] F. O. Robert Cattral, “Poker hand,” 2002.
- [18] R. S. Michalski, “A theory and methodology of inductive learning,” *Artificial Intelligence*, vol. 20, no. 2, pp. 111–161, Feb. 1983.
- [19] “The Aleph manual,” 2026, accessed: Apr. 01, 2026. [Online]. Available: <https://www.cs.ox.ac.uk/activities/programinduction/Aleph/aleph.html>